

Predicting Job Placement of College Students Using Machine Learning

Aadarsh Ganta
Virginia Tech
925 Prices Fork Road, Blacksburg, VA 24060
aadarshg@vt.edu

Abstract

This project uses machine learning to predict whether a college student will be placed in a job based on certain attributes. A logistic regression and gaussian naive bayes model were implemented in Python where each model was trained using a separate training set and then fine-tuned using stratified k-fold cross-validation. Both algorithms reached about 90% test accuracy, but naive bayes performed slightly better. The results show that CGPA, communication skills, IQ, previous semester results, and projects completed are the most important predictors of job placement. Overall, this demonstrates that simple models can capture meaningful patterns in the data and provide practical guidance to students about what areas affect employability the most. The main issue encountered was class imbalance. Although it was addressed through stratification, the issue was still evident and further tuning or different hyperparameters may need to be used to improve the predictive accuracy of these models.

1. Introduction

In today's competitive job market, many students are struggling to secure jobs after graduating. This problem has become more prominent in recent years due to the rapid advancement of AI, resulting in fewer job openings. This project investigates which factors most strongly influence job placement outcomes given information about a student such as their cumulative GPA, whether they had an internship, and how many projects they have completed. I am trying to solve a binary classification problem using logistic regression and gaussian naïve bayes model to predict whether a student will get a job or not. Several papers have tackled this problem before by implementing various other models like decision trees or random forests, but they don't address class imbalance or hyperparameter tuning in much depth. This problem is important for students and colleges because it can help them better understand what areas to focus on to improve job placement outcomes. For instance, colleges can use these insights to provide tailored support that helps students

increase their chances of getting employed by identifying what attributes influence that decision the most.

2. Relevant Work

A paper presented at the 2024 International Conference on Power, Energy, Control and Transmission Systems proposed a Placement Predictor System (PPS) designed to enhance student employability by applying data analytics and machine learning techniques [2]. The system predicted student job placement outcomes based on academic records and demographic information using data mining as well as algorithms such as logistic regression, decision trees, and random forests. The logistic regression model was very effective in predicting employability from top engineering colleges as it produced an accuracy of 87%. The system also integrated a dashboard that allowed students to view their likelihood of being placed in a job and receive personalized recommendations. PPS demonstrates a practical application of this project's objectives, providing students with valuable insights to improve career outcomes.

Another paper titled "A Review of Various Deep Learning Approaches for Placement Prediction" discusses how self-attention mechanisms can improve prediction accuracy for student job placement by understanding the relevance of certain skills [3]. It highlights how traditional machine learning models often struggle with complex and imbalanced datasets despite having high accuracy. For instance, these models fail to consider job descriptions and other important information about required skills. The paper goes into detail about some of the other limitations of deep learning models and explains how Natural Language Processing (NLP) can be applied to extract data from resumes, enhancing the accuracy of predictions. This study emphasized the importance of advancements in deep learning methods by combining self-attention mechanisms with reinforcement learning.

A study conducted by Clemente and Kwak

investigates the use of machine learning algorithms to predict job placement outcomes based on customized campus placement exams for computer science students [4]. The researchers aimed to evaluate the effectiveness of exams given by employers in predicting employment outcomes. They tested five different machine learning algorithms with all of the models achieving over 70% accuracy, but the random forest model had the highest accuracy at 90.85%. Through feature selection, it was revealed that coding-related questions were the most significant predictor of job placement. The findings from this study demonstrate that placement exams can effectively assess a student's employability and provide insights into what areas students should focus on to be successful.

3. Approach

The logistic regression and gaussian naïve bayes models implemented for this project use data from Kaggle, which contains 10,000 records of students from various colleges. The dataset contains academic and personal information about each student such as their IQ, cumulative GPA, communication skills rating, and job placement status, which is the target variable. The dataset was already clean, but I still needed to do some data preprocessing to convert the categorical variables into binary indicator columns, where each category is represented by a 0 or 1. The numeric variables were then standardized so that all features were on a similar scale. The dataset was also split into separate training and testing sets using stratified sampling, which ensures that the proportion of the two target variable classes remain the same in the train and test sets as the full dataset. Each student was represented as a vector containing the different features, which are taken as input into the models and the output is a probability of whether the student was placed in a job or not. The Kaggle notebook that was created by the owner of the dataset provided a great starting point as he developed a simple logistic regression model [1]. There were several other Kaggle notebooks that already tackled this problem, so I used the code from these when trying to preprocess the data and perform hyperparameter tuning.

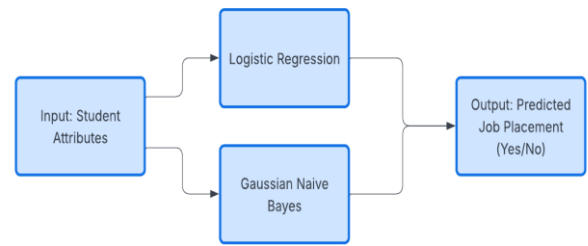


Figure 1: Model Overview Diagram

with the default hyperparameters and evaluating it on the test set. Then, I performed fine-tuning using stratified k-fold cross validation to select the best hyperparameters for the model. Finally, the tuned model was re-evaluated on the same test and compared to the baseline model. I thought this approach would be successful because logistic regression and naïve bayes models are known for performing well in binary classification problems like this one.

Other papers, which were mentioned in the relevant work section, also achieved high accuracy logistic regression, so it was a great way to compare my results to previous work. Moreover, logistic regression is easy to interpret as the coefficients can be used to analyze how each feature affects the target variable. I didn't find any papers that used naïve bayes, but I thought it was a good algorithm for this problem that could be compared to the logistic regression model since it simple and fast. In addition, a gaussian naïve bayes algorithm was used because the dataset contained mainly continuous variables. Compared to previous approaches, I explicitly handled class imbalance issues in the data since the proportion of records that indicated the student did not get placed in a job was over 60% higher than the proportion of student who got placed in a job. This is a major issue when developing predictive models as the models won't be as reliable if they are trained on skewed data. I addressed this issue by using stratified sampling to split the data and stratified cross-validation when evaluating the tuned model, which improved the accuracy of the tuned model slightly. I anticipated that these models would be successful with the default hyperparameters and that the accuracy might not get better; however, the baseline gaussian naïve bayes model performed very poorly with an accuracy of only about 50%. I needed to do some feature engineering by removing the college column, which improved the accuracy significantly. This issue may have occurred because the college column contained many different classes, which the model wasn't able to handle. On the other hand, the very first logistic regression model performed well, but I still tried improving the accuracy of both algorithms by applying regularization and tuning. This improved the logistic regression model slightly, but the naïve bayes model remained the same.

4. Experiments and Results

The dataset was split into separate training and testing sets where 70% of the data was used for training and 30% of the data was used for testing. Hyperparameter tuning was performed on the training data using stratified 5-fold cross-validation, where the data is split into five groups. Then, training is done on four of the groups while testing is done on one group, and this process is repeated five times.

Cross-validation was important for evaluating the performance of the model because it helped avoid overfitting and made sure the model generalized well. For each model, two experiments were run: the baseline model was trained on the testing set using default hyperparameters and the tuned model used grid search over defined hyperparameters with cross-validation to select the hyperparameters that maximized accuracy. Success was measured using accuracy, precision, recall, F1 score, ROC-AUC, and the confusion matrix of the test set results. The F1 score is the best measure of success because it accurately reflects performance on imbalanced data.

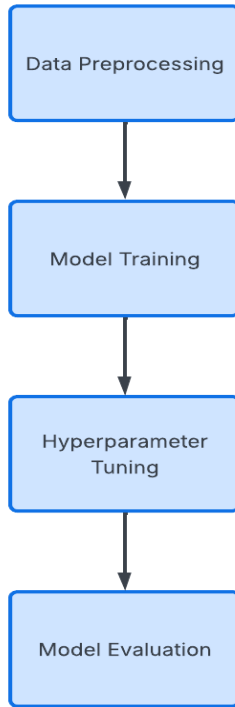


Figure 2: Workflow Diagram

4.1 Logistic Regression

For the logistic regression algorithm, the loss function is: $-(y \cdot \log(p) + (1 - y) \cdot \log(1 - p))$ where y is the true label (0 or 1) and p is the predicted probability for the positive class (1). This loss is averaged over all samples in the dataset. The main hyperparameters are the regularization strength (C) and the penalty type (L1 vs L2). The baseline model uses the default value of 1 for C and the L2 penalty. It achieved a high test accuracy of 89.93%, but an F1 score of 66.14%. The tuned model uses a range of C values including 0.01, 0.1, 1.0, 10.0, and 100.0 and either the L1 or L2 penalty. The best hyperparameters for the tuned model were a C value of 0.1 and the L1 penalty. The tuned model also uses the “liblinear” solver, which is an

optimization algorithm that minimizes the loss function. The accuracy improved slightly to 90.5% and the F1 score also improved slightly to 67.28%. The most important features were also identified using the coefficients, which determined that the communication skills rating, IQ, and cumulative GPA were the most influential factors in predicting job placement.

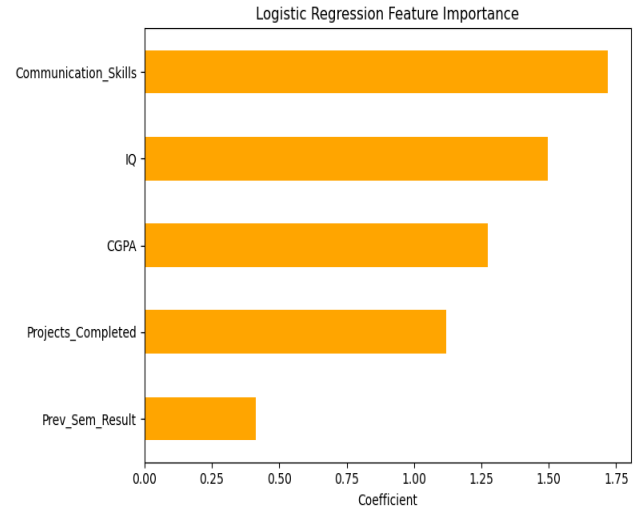


Figure 3: Feature Importance Plot for Logistic Regression

4.2 Gaussian Naïve Bayes

The loss function for the baseline gaussian naïve bayes algorithm is $-\log P(X,Y)$ where y is the true label (0 or 1) and p is the predicted probability for the positive class (1). The baseline naïve bayes model performs poorly when the college column is included, but it achieves an accuracy of 90.63% and F1 score of 69.15% when this column is removed. These values are similar to the baseline logistic regression model, but slightly better. The main hyperparameter for the gaussian naïve bayes model is variance smoothing. Hyperparameter tuning is performed over a log grid with variance smoothing values from 10^{-12} to 10^{-4} using stratified 5-fold cross-validation. The best variance smoothing value for the tuned model was 10^{-12} , but the accuracy and F1 scores remain the same even after tuning. The model didn’t overfit, but it generalized moderately well. The most important features were also identified by calculating the mean differences using the means that are stored in the threat attribute. It was determined that the communication skills rating, cumulative GPA, and the previous semester GPA were the most influential factors in predicting job placement.

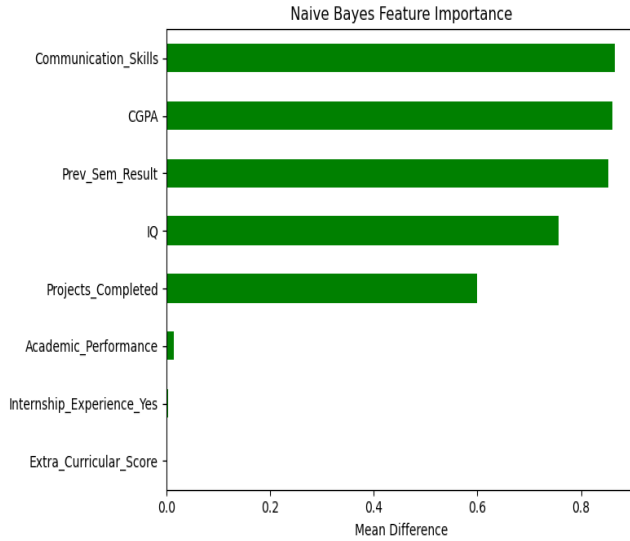


Figure 4: Feature Importance Plot for Naïve Bayes

4.3 Results

Between the two algorithms, the naïve bayes performed slightly better. Despite achieving high accuracies for both algorithms, I think I was moderately successful in predicting job placement as the results may have been affected by class imbalance. The F1 score was around 65% to 70% for both algorithms, which is decent, but it could be better. However, I was successfully able to determine what features impact job placement status the most, which provide useful insights. Figure 5 below shows all of the evaluation metrics, displaying the highest value from each either naïve bayes (NB) or logistic regression (LR). From this table, we can see that naïve bayes are better in terms of recall and F1 score while logistic regression is better in terms of precision and ROC-AUC.

Evaluation Metric	Formula	Highest Value
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	0.91 (NB)
Precision	$TP / (TP + FP)$	0.79 (LR)
Recall	$TP / (TP + FN)$	0.63 (NB)
F1	Harmonic mean of precision and recall	0.69 (NB)
ROC-AUC	Area under ROC curve	0.94 (LR)

Figure 5: Table of Evaluation Metrics

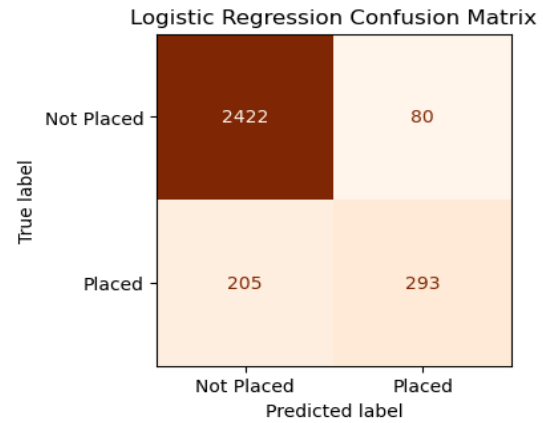


Figure 6: Confusion Matrix for Logistic Regression

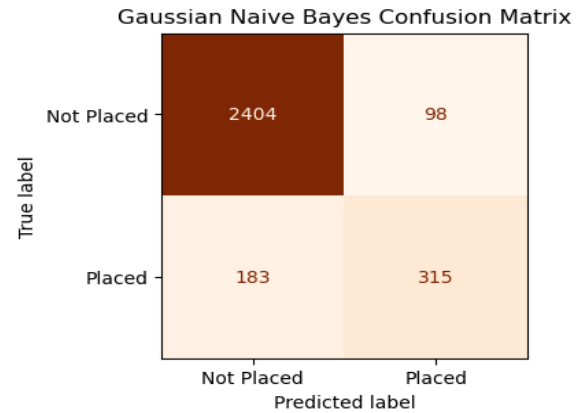


Figure 7: Confusion Matrix for Logistic Regression

5 Availability and Reproducibility

The complete code used for this project is available as a Jupyter notebook in a public Git repository, and I used an MIT open-source license to release the code [9]. The CSV file containing the dataset is also included in the Git repository. The implementation uses open-source libraries in python such as sci-kit learn and pandas, and the code includes the preprocessing steps, the baseline and tuned models for both algorithms, and the evaluation and visualization code. The training and testing split as well as the model parameters were created using a fixed random seed, so others can produce the same results. The confusion matrices in figure 6 and 7 demonstrate that the models are good at predicting non-placed students, but not as effective at predicting placed students. This points back to the class imbalance issue as

6. Conclusion and Future Work

This project showed that job placement for college students can be predicted reasonably well using simple, interpretable algorithms like logistic regression and naïve bayes. These algorithms achieved around 90% test accuracy with a reasonable F1 score of around 65%. Both models were easily interpretable as the most important features can clearly be observed, highlighting that students should improve their communication skills, GPA, and complete more projects to enhance their chances of getting a job after graduating. The significance of this problem extends beyond just students as well. Colleges can leverage insights from these predictive models to identify which academic or personal factors most influence career success, allowing them to refine curriculum, add additional courses, and provide targeted support for students. Ultimately, this project contributes to an area that continues to change, requiring the need for more data and models that can help people understand how to adapt.

This project could be further improved by implementing additional models such as decision trees or random forests to see whether these methods improve performance. Additionally, more features could be added to the dataset to explore more specific metrics that influence job placement. More fine-tuning can also be done to address the class-imbalance issue and improve the F1 score. Lastly, testing these models on various datasets can help understand how the model generalizes and whether it can be applied to make real-world predictions. These models will soon be used in the workplace to improve decision-making for institutions, companies, and students who are navigating an increasingly competitive corporate landscape.

References

- [1] Sumedh1507. College Student Placement. Kaggle. Retrieved October 15, 2025, from <https://www.kaggle.com/datasets/sumedh1507/college-student-placement/data>
- [2] M. K. J., V. M., T. P. Rani, & S. S. S. (2024). Comprehensive Career Placement Predictor: An Analytical Tool for Optimizing Job Placement Outcomes. 2024 International Conference on Power, Energy, Control and Transmission Systems (ICPECTS), Chennai, India, 1–6. <https://doi.org/10.1109/ICPECTS62210.2024.10780012>
- [3] M. Bodke, V. Tiwari, J. Wadhwani, T. Gorbade, & R. Soundane. (2025). A Review of Various Deep Learning Approaches for Placement Prediction. 2025 International Conference on Machine Learning and Autonomous Systems (ICMLAS), Prawet, Thailand, 179–184. <https://doi.org/10.1109/ICMLAS64557.2025.10968796>
- [4] Clemente, C. J., & Kwak, M. (2022). Utilizing Data Science and Analytics in Predicting Campus Placement. *Issues in Information Systems*. https://doi.org/10.48009/3_iis_2022_106
- [5] Syed, M. K. Logistic Regression with Hyperparameter Tuning. Kaggle. Retrieved December 10, 2025, from <https://www.kaggle.com/code/muhammadransyad/logistic-regression-with-hyperparameter-tuning>
- [6] GeeksforGeeks. How to Optimize Logistic Regression Performance. Retrieved December 10, 2025, from <https://www.geeksforgeeks.org/machine-learning/how-to-optimize-logistic-regression-performance/>
- [7] Nalayani, N. C. M., Akilan, N. T., V. S., N. H., SaranArulnathan, N., & S. N. V. (2023). Placement Analysis for Students Using Machine Learning. *Journal of Information Technology and Digital World*, 5(3), 223–237. <https://doi.org/10.36548/jitdw.2023.3.001>
- [8] Lucidchart. Flowchart. Retrieved December 10, 2025, from <https://lucid.app/lucidchart/3fbc2e79-0a66-4b17-837a-890adc001f76/edit>
- [9] Aadarshganta. Student Job Placement Prediction Using Machine Learning. GitHub. Retrieved December 10, 2025, from <https://github.com/aadarshganta/Student-Job-Placement-Prediction-Using-Machine-Learning>